CLOUD ENGINEERING CIRCLE 15 GROUP PROJECT -

EXPENSE TRACKER WEB APPLICATION ON AWS

1. PROJECT OVERVIEW

Project Title

Building and Deploying a Serverless Expense Tracker on AWS

Project Objective

The objective of this project is to build a simple web-based expense tracking application and deploy it on Amazon Web Services (AWS).

The application will allow users to create accounts, log in securely, record their expenses, view their expense history, calculate their total spending, and delete expenses.

The project will demonstrate practical understanding of cloud computing, serverless architecture, application development, authentication, database management, API integration, cloud deployment, security, and monitoring.

Project Scope

The application will include:

- User registration (Sign Up).
- User authentication (Login).
- Email verification.
- User logout.
- Expense creation.
- Expense history.
- Total expense calculation.
- Expense deletion.

- User-specific expense records.
- Cloud deployment with a publicly accessible HTTPS URL.

2. TECHNOLOGIES AND TOOLS

Development Tools

Tool	Purpose
VS Code	Code editor
Git	Version control
GitHub	Shared repository and team collaboration
AWS CLI	Interact with AWS through the terminal
HTML	Frontend structure
CSS	Frontend styling
JavaScript	Frontend functionality and API integration
Python	Backend development
Browser	Application testing
Postman	API testing

3. APPLICATION ARCHITECTURE

Architecture Overview

The application will use a serverless architecture.

Users will access the website through a browser. The frontend will be hosted on Amazon S3 and delivered through CloudFront.

Amazon Cognito will handle registration and authentication.

Authenticated users will interact with the backend through API Gateway. API Gateway will invoke Lambda functions, which will process expense operations and communicate with DynamoDB.

Architecture Flow

User / Browser
↓
Amazon CloudFront
↓
Amazon S3 (Frontend Hosting)

Authentication:
Frontend → Amazon Cognito → Authentication Tokens

Expense Operations:
Frontend → API Gateway → Lambda → DynamoDB

Monitoring:
AWS Lambda → Amazon CloudWatch

AWS Infrastructure Required

1. Amazon S3

Purpose:

- Store HTML, CSS, and JavaScript files.

- Serve as the origin for CloudFront.

2. Amazon CloudFront

Purpose:

- Deliver the frontend.
- Provide HTTPS access.
- Cache frontend files for faster delivery.

3. Amazon Cognito

Purpose:

- Register users.
- Authenticate users.
- Verify email addresses.
- Manage passwords.
- Issue authentication tokens.

4. Amazon DynamoDB

Purpose:

- Store expense records.
- Retrieve user expenses.
- Support expense deletion.

5. AWS Lambda

Purpose:

- Add expenses.
- Retrieve expenses.
- Delete expenses.
- Calculate total expenses.
- Enforce user-specific data access.

6. Amazon API Gateway

Purpose:

- Expose backend endpoints.
- Connect frontend requests to Lambda.
- Protect expense APIs using Cognito authentication.

7. AWS IAM

Purpose:

- Grant Lambda access to DynamoDB.
- Manage AWS resource permissions.
- Restrict access to project resources.

8. Amazon CloudWatch

Purpose:

- Store Lambda logs.
- Monitor execution.
- Troubleshoot errors.

4. DYNAMODB DATABASE DESIGN

Table Name

Expenses

Primary Key Configuration

Partition Key:

- Name: userId
- Type: String

Sort Key:

- Name: expenseId
- Type: String

Billing Mode:

- On-demand

Expense Record Structure

Attribute	Type	Description
userId	String	Cognito user identifier
expenseId	String	Unique expense identifier
amount	Number	Amount spent
category	String	Expense category
description	String	Expense description
expenseDate	String	Date expense occurred
createdAt	String	Timestamp expense was created

Example Expense Records

Record 1:

{
"userId": "USER-A",
"expenseId": "EXP-001",
"amount": 5000,
"category": "Food",
"description": "Lunch",
"expenseDate": "2026-11-03",
"createdAt": "2026-11-03T13:00:00Z"
}

Record 2:

{
"userId": "USER-A",
"expenseId": "EXP-002",
"amount": 2000,
"category": "Transport",
"description": "Bus fare",
"expenseDate": "2026-11-03",
"createdAt": "2026-11-03T15:00:00Z"
}

Why userId is the Partition Key

The partition key identifies the user who owns an expense.

Using the Cognito user's unique identifier allows expenses to be grouped by user.

Why expenseId is the Sort Key

The sort key uniquely identifies each expense belonging to a user.

The combination of userId and expenseId uniquely identifies every expense record.

Important Database Design Consideration

The expenseDate attribute will store the actual expense date.

The expenseId will be generated automatically by Lambda using a UUID.

For the initial version, expenses can be retrieved and sorted by expenseDate in the application.

5. AUTHENTICATION DESIGN

Amazon Cognito User Pool

Create one Cognito User Pool for the application.

Configuration

- Sign-in identifier: Email.
- Registration: Email-based.
- Email verification: Enabled.
- Password authentication: Enabled.
- Password policy: Use reasonable password requirements.
- App Client: Create an app client for the frontend.
- Authentication flow: Use Cognito's managed authentication or SDK-supported authentication.

Authentication Workflow

Sign Up

1. User opens the Sign Up page.
2. User enters name, email, and password.
3. Frontend sends registration request to Cognito.
4. Cognito creates the account.
5. Cognito sends an email verification code.
6. User verifies email.
7. User can log in.

Login

1. User enters email and password.
2. Frontend sends authentication request to Cognito.
3. Cognito verifies credentials.
4. Cognito returns authentication tokens.
5. Frontend uses the access token to access protected APIs.

Logout

1. User clicks Logout.
2. Frontend signs the user out.
3. Authentication session is cleared.
4. User is redirected to Login.

Important Authentication Rule

Passwords must not be stored in DynamoDB.

Cognito manages passwords and authentication.

6. BACKEND DESIGN

AWS Lambda

The backend will be implemented using Python.

The Lambda function will receive requests from API Gateway, process them, and communicate with DynamoDB.

For a beginner-friendly implementation, one Lambda function can handle the initial expense operations using the HTTP method and route.

Alternatively, separate Lambda functions can be created for each operation.

Backend Responsibilities

- Validate incoming requests.
- Authenticate requests through API Gateway's authorizer.
- Identify the authenticated user.
- Create expense records.
- Retrieve expense records.
- Delete expense records.

- Calculate total spending.
- Return appropriate responses.
- Handle errors.

User Identification

The authenticated user's Cognito sub will be used as userId.

Lambda must extract the authenticated user's identity from API Gateway's validated authorizer context.

The frontend must never be trusted to supply the owner userId.

7. FRONTEND DESIGN

The frontend will be built using HTML, CSS, and JavaScript.

Required Pages

1. Sign Up Page

Fields:

- Full Name.
- Email.
- Password.
- Confirm Password.

Features:

- Create account.
- Email verification.
- Redirect to Login.

2. Login Page

Fields:

- Email.
- Password.

Features:

- Login.
- Error messages.
- Redirect to Dashboard.
- Forgot password functionality if time permits.

3. Expense Dashboard

The dashboard should contain:

- Total Expenses.
- Add Expense button.
- Expense History.
- Expense Categories.
- Delete Expense button.
- Logout button.

4. Add Expense Form

Fields:

- Amount.
- Category.
- Description.
- Expense Date.

Categories:

- Food.
- Transport.
- Bills.

- Shopping.
- Entertainment.
- Other.

5. Expense History

Display:

- Expense description.
- Category.
- Amount.
- Date.

Users should be able to delete individual expenses.

8. TEAM TASK ALLOCATION

Frontend - Ibrahim & Nonso

Responsibilities:

- Build Sign Up page.
- Build Login page.
- Build Dashboard.
- Build Add Expense form.
- Build Expense History.
- Implement frontend interactions.
- Integrate frontend with Cognito.
- Integrate frontend with API Gateway.

Deliverables:

- Frontend code.
- Working authentication interface.
- Working expense dashboard.

Backend - Cesar, Adedamola & Peter

Responsibilities:

- Create Lambda function.
- Write Python backend code.
- Implement Add Expense.
- Implement Get Expenses.
- Implement Delete Expense.
- Implement expense calculations.
- Handle request validation.
- Handle backend errors.

Deliverables:

- Lambda code.
- Working backend operations.
- Backend testing results.

Database and API - Everyone

Responsibilities:

- Create DynamoDB table.
- Configure partition and sort keys.
- Test DynamoDB operations.
- Create API Gateway.
- Configure routes.
- Connect API Gateway to Lambda.
- Configure Cognito JWT authorizer.
- Test API endpoints.

Deliverables:

- DynamoDB table.
- API Gateway configuration.

- Working API endpoints.

AWS Deployment and Infrastructure - Everyone

Responsibilities:

- Coordinate AWS environment setup.
- Configure IAM permissions.
- Create S3 bucket.
- Configure CloudFront.
- Deploy frontend.
- Coordinate backend deployment.
- Monitor CloudWatch logs.
- Test public application URL.
- Maintain deployment documentation.

Deliverables:

- Deployed frontend.
- CloudFront URL.
- AWS infrastructure documentation.
- Deployment screenshots.

9. GITHUB COLLABORATION WORKFLOW

Suggested Repository Structure

```
expense-tracker-aws/
│
├── frontend/
│   ├── index.html
│   ├── login.html
│   ├── signup.html
│   ├── dashboard.html
│   ├── css/
```

```
│ └── js/
│
├── backend/
│ ├── lambda_function.py
│ └── requirements.txt
│
├── docs/
│ ├── architecture.md
│ └── screenshots/
│
└── README.md
```

Branches

- main
- frontend
- backend
- database-api
- deployment

Collaboration Workflow

1. Repository owner creates the GitHub repository.
2. All members are added as collaborators.
3. Each member works on their assigned branch.
4. Members commit and push changes.
5. Members create pull requests.
6. Team reviews changes.
7. Changes are merged into main.
8. Integrated application is deployed.

Never commit passwords, AWS access keys, or secret credentials to GitHub.

10. AWS IMPLEMENTATION SEQUENCE

The following sequence should be followed during implementation.

Step 1: AWS Account and Access

- Choose the AWS account that will host the project.
- Configure access for group members.
- Use individual authorized logins.
- Select a project region.
- Establish resource naming conventions.
- Set a billing budget or alert.

Step 2: Create DynamoDB

- Create Expenses table.
- Set userId as partition key.
- Set expenseId as sort key.
- Use String for both keys.
- Choose on-demand billing.
- Test creating and retrieving records.

Step 3: Create Cognito

- Create User Pool.
- Configure email sign-in.
- Configure email verification.
- Create App Client.
- Connect frontend to Cognito.
- Test Sign Up and Login.

Step 4: Create Lambda

- Create Lambda function.
- Select Python runtime.
- Configure execution role.
- Grant DynamoDB access.

- Write backend logic.
- Test Lambda using sample events.

Step 5: Create API Gateway

- Create HTTP API.
- Configure routes.
- Integrate routes with Lambda.
- Configure Cognito JWT authorizer.
- Protect expense endpoints.
- Configure CORS.
- Test endpoints.

Step 6: Integrate Frontend

- Connect registration to Cognito.
- Connect login to Cognito.
- Connect Dashboard to API Gateway.
- Send authentication tokens.
- Test expense creation.
- Test expense retrieval.
- Test expense deletion.

Step 7: Deploy Frontend

- Create S3 bucket.
- Upload frontend files.
- Configure CloudFront.
- Configure default root object.
- Wait for distribution deployment.
- Test public HTTPS URL.

Step 8: Final Integration

- Test Sign Up.

- Test email verification.
- Test Login.
- Test Add Expense.
- Test Expense History.
- Test Delete Expense.
- Test Logout.
- Verify user-specific expense access.
- Troubleshoot errors.
- Capture screenshots.

11. SECURITY REQUIREMENTS

The application must follow these security principles:

1. Do not store user passwords in DynamoDB.
2. Use Cognito for authentication.
3. Protect expense API endpoints using a Cognito JWT authorizer.
4. Use the authenticated user's Cognito sub as the expense owner.
5. Do not trust userId supplied by the frontend.
6. Ensure users cannot retrieve or delete other users' expenses.
7. Grant Lambda only the DynamoDB permissions it needs.
8. Do not expose AWS credentials in frontend code.
9. Do not commit secrets to GitHub.
10. Use HTTPS for the deployed website.
11. Configure appropriate CORS restrictions.
12. Avoid exposing unnecessary AWS resources publicly.

12. TESTING CHECKLIST

<u>Authentication</u>

- User can register.
- User receives verification email.
- User can verify email.

- User can log in.
- Invalid credentials are rejected.
- User can log out.

Expense Operations

- User can add an expense.
- Expense is saved in DynamoDB.
- User can retrieve expenses.
- Total expenses are calculated correctly.
- User can delete an expense.
- Deleted expense no longer appears.

Security

- Unauthenticated users cannot access expense APIs.
- User A cannot retrieve User B's expenses.
- User A cannot delete User B's expenses.
- Frontend cannot override expense ownership.

Deployment

- Frontend loads through CloudFront.
- HTTPS works.
- Sign Up works on deployed website.
- Login works on deployed website.
- Expense operations work on deployed website.
- Application works on different browsers.

13. PROJECT DELIVERABLES

At the end of the project, we should have:

1. A functional Expense Tracker application.
2. A working Sign Up and Login system.
3. A DynamoDB database containing expense records.
4. A deployed backend using Lambda.
5. API endpoints protected by Cognito.
6. A publicly accessible HTTPS website.
7. A GitHub repository containing project code.
8. An AWS architecture diagram.
9. Screenshots of AWS infrastructure.
10. A project README.
11. A presentation explaining the architecture.
12. Demonstration of the application.